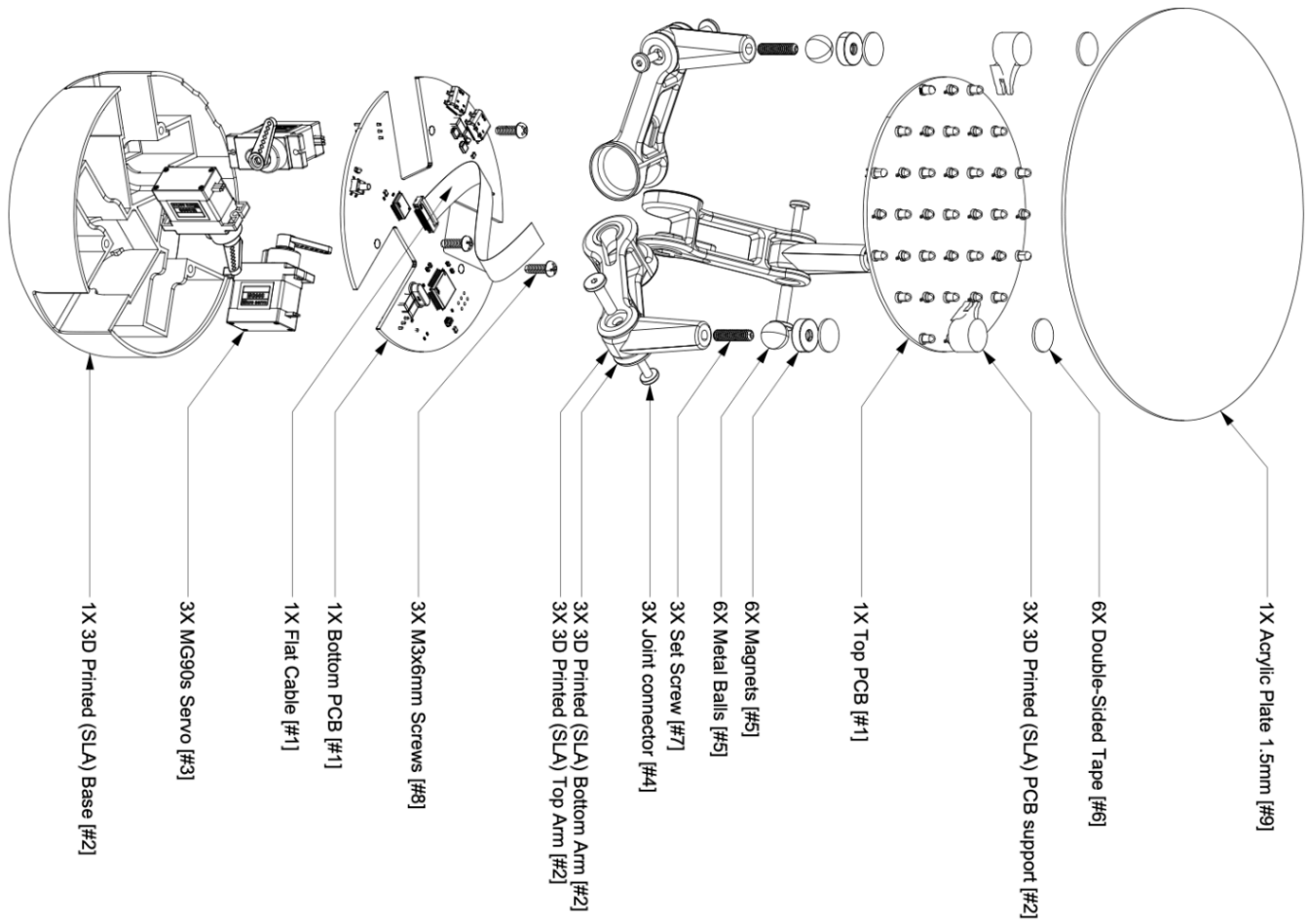
How BaBot Works



Introduction

Congratulations on building your BaBot! You've assembled a sophisticated control system that balances a ping pong ball using the same principles found in drones, self-driving cars, and industrial robots.

But how does it actually work?

Imagine this: You throw a ping pong ball onto BaBot's transparent platform. It lands off-center, rolling toward the edge. In about 25 milliseconds, BaBot detects it, calculates its position, determines the perfect correction, and tilts the platform—catching the ball before it can escape. This happens ~40 times every second, creating seamless, seemingly effortless balance.

What looks like magic is actually a precisely orchestrated sequence of sensing → calculation → control → motion. And now that you've built BaBot, you're ready to understand exactly what's happening in each control cycle.

This guide will walk you through each step, from the moment the ball touches the platform to the servo movements that bring it back to center. We'll explore:

- PID control—the brain of every feedback system
- Inverse kinematics and the 3-servo tilting mechanism
- How it all runs in a continuous 40 Hz feedback loop

By the end, you'll understand not just *what* BaBot does, but *why* it works—and how you can experiment to make it even better.

Let's begin the journey.

Step 1: Detecting the Ball

The Challenge

The moment a ping pong ball lands on BaBot's transparent platform, the first challenge is simple but critical: where exactly is the ball?

BaBot needs to know the ball's position with millimeter precision, and it needs to know it fast—40 times per second. To do this, BaBot uses an invisible grid of light beneath the platform.

IR Light: Seeing the Invisible

IR (Infrared) light is light that's just beyond what human eyes can see. While we can see colors from red to violet, IR light has a wavelength slightly longer than red light—making it invisible to us, but perfect for sensors.

Why use IR instead of visible light? IR light is invisible to you, so BaBot can constantly shine bright light without being distracting.

The Sensing System

Beneath BaBot's transparent PMMA platform, there's a carefully designed detection system with three key components:

21 IR LEDs distributed across the plate emit invisible infrared light upward toward the platform.

16 IR Phototransistors arranged in a 4×4 grid detect how much IR light reflects back down.

The transparent platform made of PMMA lets IR light pass through freely in both directions.

What's a Phototransistor?

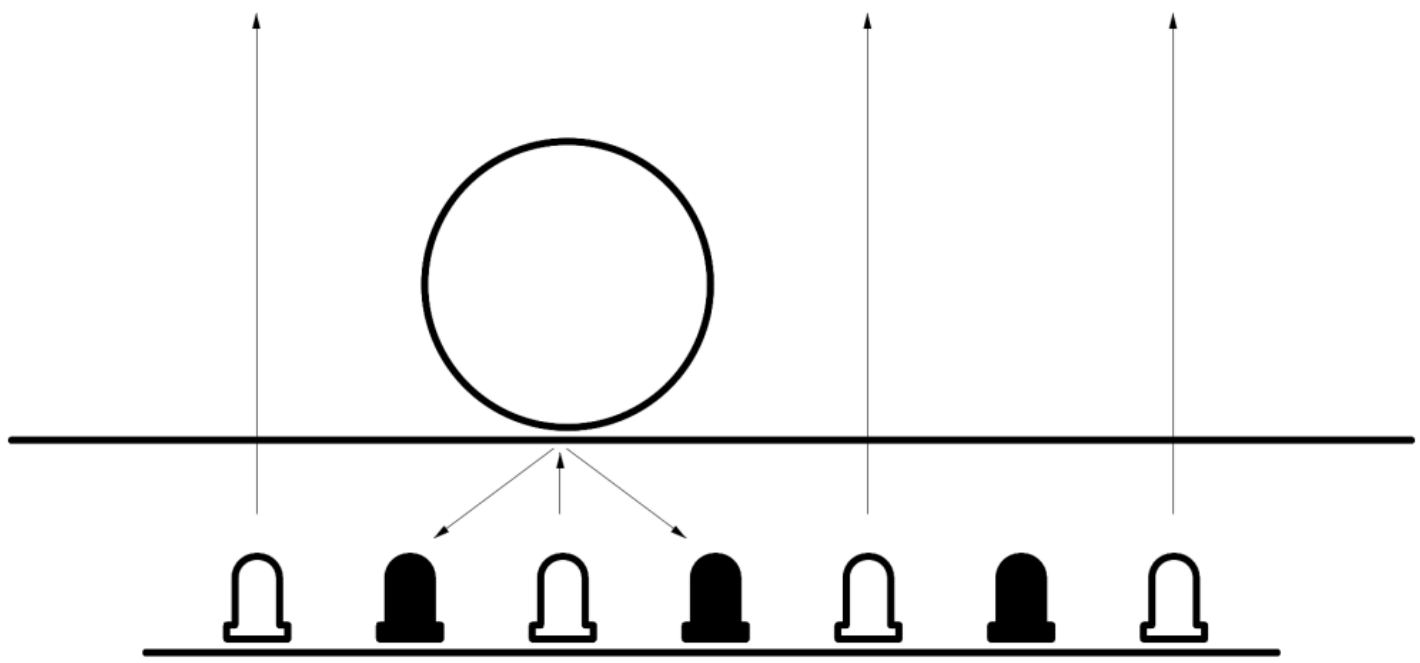
A phototransistor is like a light-sensitive switch. The more light hits it, the more electrical current flows through it. Think of it as a sensor that converts light intensity into a number BaBot can read.

When the IR LEDs shine upward and there's no ball, the light mostly passes through the transparent platform or scatters away—the phototransistors see

How the Ball Creates a Bright Spot

The smooth surface of the ping pong ball acts like a mirror for IR light. When a ball is present, it reflects the IR light back down toward the sensors, creating a bright spot in the sensor readings directly beneath the ball.

Think of it this way: without a ball, the IR light shoots upward and disperses. With a ball, the IR light bounces off the bottom of the ball and comes straight back down to the sensors below it. The closer a sensor is to being directly under the ball, the brighter its reading.



Removing Noise: Ambient Light Subtraction

Here's a problem: room lights, sunlight through windows, and even your phone screen all emit some IR light. How does BaBot know what's the ball and what's just background?

BaBot uses a clever two-step measurement for each sensor:

Step 1: Lights OFF – Turn off the 21 IR LEDs and measure each sensor. This captures the ambient background IR light in the room.

Step 2: Lights ON – Turn on the IR LEDs and measure again. Now the sensors see ambient light plus the ball's reflection.

Step 3: Subtract – The actual signal from the ball is simply: Lights ON reading minus Lights OFF reading.

This subtraction leaves only the ball's reflection. By removing the background, BaBot only "sees" the ball.

Here's what this looks like in the code:

```
// Measure ambient light (LEDs off)
digitalWrite(IR_LED_PIN, LOW);
```

```
// Measure with IR LEDs on
digitalWrite(IR_LED_PIN, HIGH);
for (int i = 0; i < 16; i++) {
  irLight[i] = analogRead(sensor[i]);
}

// Calculate the actual signal from the ball
for (int i = 0; i < 16; i++) {
  irSignal[i] = irLight[i] - ambientLight[i];
}
```

From 16 Numbers to Ball Position

Now BaBot has 16 sensor readings arranged in a 4×4 grid. Each number represents how much IR light that sensor detected. Sensors directly under the ball will have high values, while sensors far from the ball will have low values.

The Weighted Average

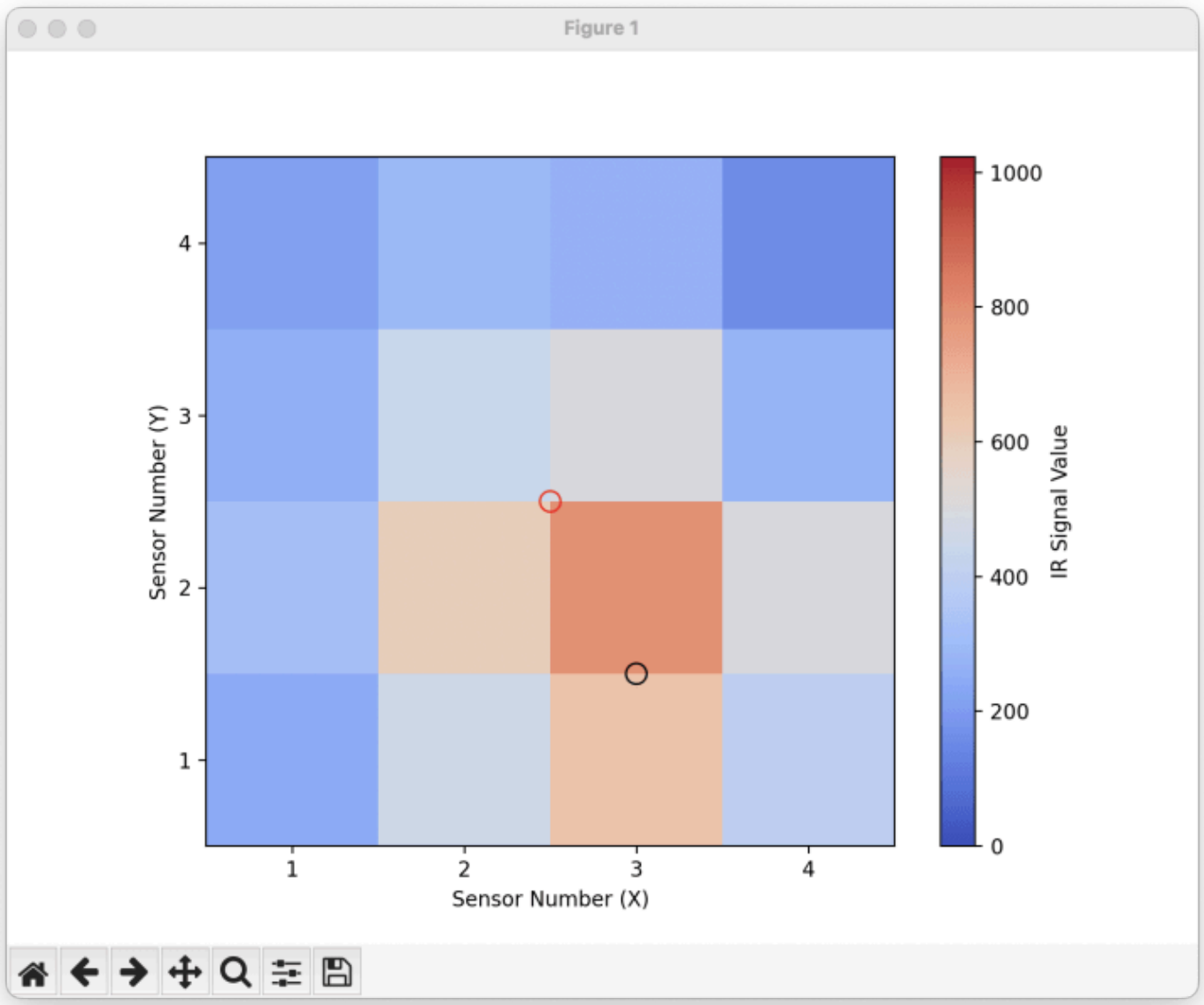
Imagine the sensor grid like a digital scale with 16 weight sensors. If you place something heavy on one corner, that corner's sensor reads high. To find the center of weight, you'd take an average weighted by each sensor's reading.

BaBot does the same thing with light. The basic idea is:

X position = (sum of each sensor's X coordinate times its brightness) divided by (total brightness)

This is called a weighted centroid—it finds the “center of brightness.”

Here’s what this means in practice. If the ball is in the center, the four central sensors read bright while outer sensors read dim, so the average position comes out to the center. If the ball moves right, the right sensors get brighter and left sensors get dimmer, so the average position shifts right. Even when the ball is between sensors, the weighted average finds the exact position.



Step 2: PID Control – The Brain of BaBot

The Problem

BaBot now knows exactly where the ball is. Let’s say it’s 2 centimeters to the right of center. The question is: how much should BaBot tilt the platform to bring it back?

This might seem simple at first. If the ball is 2 cm to the right, just tilt left by some amount. But here’s the catch: if you tilt too little, the ball never quite makes it back to center. If you tilt too much, the ball overshoots, rolls past center to the other side, and now you have the opposite problem. You could end up with the ball oscillating back and forth forever, or worse, flying off the edge.

This is where PID control comes in. PID is a mathematical algorithm that calculates the perfect correction based on three different pieces of

Batch #3 is here, limited kits Got it!

countless other control systems.

What is PID?

PID stands for Proportional-Integral-Derivative. Each of these three terms looks at the error (the difference between where the ball is and where you want it to be) in a different way, and together they create smooth, stable control.

The PID equation is:

$$\text{Output} = K_P \cdot e(t) + K_I \cdot \int e(t)\,dt + K_D \cdot \frac{de(t)}{dt}$$

Let's break down what each term does.

The P Term: Proportional

The proportional term is the most intuitive. It says: the farther the ball is from center, the more I should tilt.

$$e(t) = \text{target position} - \text{current position}$$

If the ball is 3 cm away from center, you tilt more than if it's only 1 cm away. The K_P gain is just a multiplier that controls how aggressive this response is.

$$P_{\text{term}} = K_P \cdot e(t)$$

In BaBot's code, $K_P = 3.0$ This means if the ball is 1 cm off center, the P term contributes 3 degrees of tilt. If the ball is 2 cm off, it contributes 6 degrees.

The D Term: Derivative

The derivative term looks at how fast the error is changing. It's about velocity—how quickly is the ball moving?

$$\frac{de(t)}{dt} = e(t) - e(t-1) = \text{rate of change of error}$$

If the ball is racing toward the center at high speed, you need to start tilting the other way now to slow it down, otherwise it'll overshoot. The derivative term predicts where the ball is going and acts like a brake.

$$D_{\text{term}} = K_D \cdot \frac{de(t)}{dt}$$

In BaBot, $K_D = 30.0$, which is much larger than the P gain. This strong derivative action is what prevents oscillation. Without it, the ball would bounce back and forth across the center like a pendulum. The D term dampens this motion, bringing the ball smoothly to rest at the center.

Here's the key insight: when the ball is moving quickly toward center, the error is decreasing rapidly. This makes $\frac{de(t)}{dt}$ negative, which means the D term opposes the motion, creating a braking effect.

The I Term: Integral

Here's the problem with using only P and D control: imagine the ball is sitting slightly off-center, maybe because the platform isn't perfectly level or there's a

it can't overcome friction or that tiny breeze, and the ball settles at a position that's slightly off-center. It never quite makes it all the way. This is called steady-state error.

The integral term solves the steady-state error problem. It keeps track of how long the ball has been off-center.

$$\int e(t)\,dt = \text{accumulated error over time}$$

Think of it like this: if the ball has been sitting 1 cm to the right for several seconds, even though the error is small, the fact that it's been there for a while means something is wrong. Maybe there's a constant disturbance you need to counteract. The integral term accumulates this error over time.

$$I_{\text{term}} = K_I \cdot \int e(t)\,dt$$

In BaBot, $K_I = 0.02$, which is quite small. Every control cycle, BaBot adds the current error to a running sum. If the ball stays off-center, this sum grows and grows, adding more and more tilt until the ball finally moves back to center. Once it reaches center, the accumulated error stops growing.

The integral term is what eliminates drift and ensures the ball actually settles at exactly center, not "close to center."

Putting It All Together

At every control cycle (40 times per second), BaBot calculates all three terms and adds them up:

$$\theta_{\text{roll}} = K_P \cdot e_x + K_I \cdot \sum e_x + K_D \cdot \Delta e_x$$
$$\theta_{\text{pitch}} = K_P \cdot e_y + K_I \cdot \sum e_y + K_D \cdot \Delta e_y$$

BaBot runs this calculation separately for the X axis (roll) and Y axis (pitch), so it can handle a ball moving diagonally.

Tuning: The Art of PID

The values $K_P = 3.0$, $K_I = 0.02$, and $K_D = 30.0$ aren't random. They've been carefully tuned to give BaBot the right balance of speed and stability. But here's the exciting part: you can change them.

If you increase K_P, BaBot will react more aggressively to errors. The ball will return to center faster, but it might oscillate more.

If you increase K_I, steady-state errors will disappear faster, but too much I can cause the platform to "wind up" and overshoot dramatically.

If you increase K_D, oscillations will be damped more, making motion smoother, but too much D can make BaBot too agressive.

Finding the perfect balance is part of the art and science of control systems. And now that you've built BaBot, you can experiment with these values yourself and see how they affect the behavior in real time.

The Output

Batch #3 is here, limited kits Got it!

be something like "tilt 4 degrees in roll, 2 degrees in pitch.

But BaBot doesn't have a roll motor and a pitch motor. It has three servos arranged in a triangle. How does it convert these two angles into three servo positions?

That's where inverse kinematics comes in.

Step 3: Inverse Kinematics – From Angles to Servos

The Challenge

PID control has given BaBot two numbers: roll angle and pitch angle.

But here's the problem: BaBot doesn't have a roll motor and a pitch motor. Instead, it has three servo motors arranged in a triangle, each controlling one arm. These three servos must work together to create any desired tilt.

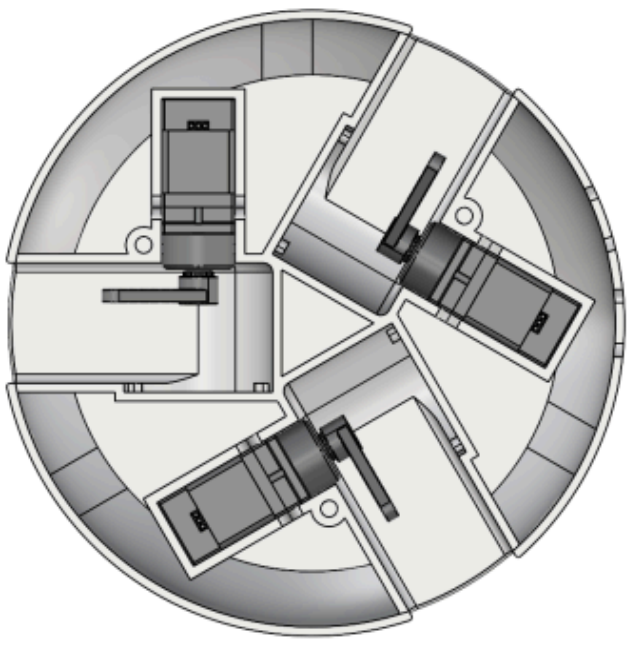
This is the challenge of inverse kinematics: given a desired platform orientation (roll and pitch), calculate the exact angle each of the three servos needs to be at.

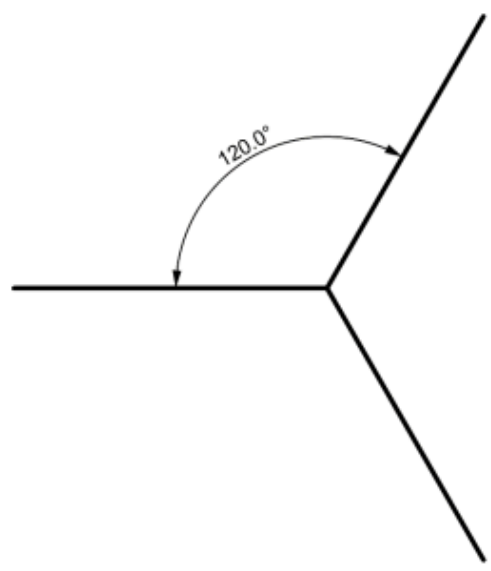
The 3-DOF Parallel Mechanism

BaBot uses what's called a 3-DOF (three degrees of freedom) parallel mechanism. Three servo motors are positioned around a circular base, spaced exactly 120° apart. Each servo controls an arm made of two segments:

- The lower arm (50 mm long) connects to the servo
- The upper arm (39.2 mm long) connects to the platform

The three servos are located at angles 0°, 120°, and 240° around the base circle. The platform attachments are also 120° apart.





When all three servos are at the same angle, the platform is level. When they move to different angles, the platform tilts. The key is figuring out exactly

Batch #3 is here, limited kits Got it!

Imagine you want to tilt the platform 5° to the right. Which way should each of the three servos move? And by how much?

This isn't obvious because the servos aren't aligned with roll and pitch axes. They're arranged in a triangle, 120° apart. When you tilt right, all three servos have to move—but each one moves a different amount and in a coordinated pattern.

Forward kinematics is easy: if you know the three servo angles, you can figure out how the platform tilts. But inverse kinematics goes backwards: you know how you want the platform to tilt, and you need to figure out the three servo angles. That's much harder.

The Solution: Work Backwards Through Geometry

Here's how BaBot solves this problem:

Step 1: Figure out where the platform attachments end up

Imagine the platform starts level at 65 mm height. Three attachment points sit evenly around the platform circle. Now you want to tilt it by your desired roll and pitch angles.

BaBot uses rotation math to figure out where each of those three attachment points ends up in 3D space after the tilt. This involves some trigonometry (sine and cosine functions), but the idea is simple: rotate each point around the center by the roll angle, then by the pitch angle.

After this step, BaBot knows the exact 3D coordinates of all three platform attachment points.

Step 2: Calculate each servo angle

Now BaBot has a geometry puzzle for each servo. It knows:

- Where the servo sits (at the base)
- Where the platform attachment point is (from Step 1)
- The two arm lengths (50 mm and 39.2 mm)

The question is: what angle does the servo need to be at to connect these two points with those two arm lengths?

Think of it like this: you're trying to reach from point A to point B using two rigid sticks. There's only one way to bend your "elbow" to make it work. BaBot uses the law of cosines—a geometry rule about triangles—to find that angle.

Step 3: Command the servos

BaBot calculates this angle for all three servos independently. Each servo gets its own unique angle. Then it sends all three angles to the servo motors, which move to those positions.

The platform tilts to exactly the orientation PID control requested.

forward/back (pitch).

BaBot calculates where the three platform attachments end up:

- Servo A (at 0°): its attachment point moves slightly down
- Servo B (at 120°): its attachment point moves up and to the side
- Servo C (at 240°): its attachment point also moves up and to the side

Then it calculates:

- Servo A needs to rotate to 72°
- Servo B needs to rotate to 68°
- Servo C needs to rotate to 68°

All three servos move to those angles. The platform tilts 4° to the right. The ball experiences a slight slope pulling it back toward center.

From Request to Reality

The complete sequence happens in milliseconds:

1. PID control outputs: "tilt 5° in roll, 3° in pitch"
2. Inverse kinematics calculates: "Servo A → 70°, Servo B → 68°, Servo C → 72°"
3. Servos move to those positions
4. Platform tilts exactly as requested
5. Ball rolls back toward center

This calculation happens 40 times per second. Each time, BaBot measures where the ball is, PID decides how to tilt, inverse kinematics converts that to servo angles, and the servos move.

That's how BaBot balances a ball—a continuous loop of sensing, thinking, and acting.

Copyright © 2025 All Rights Reserved